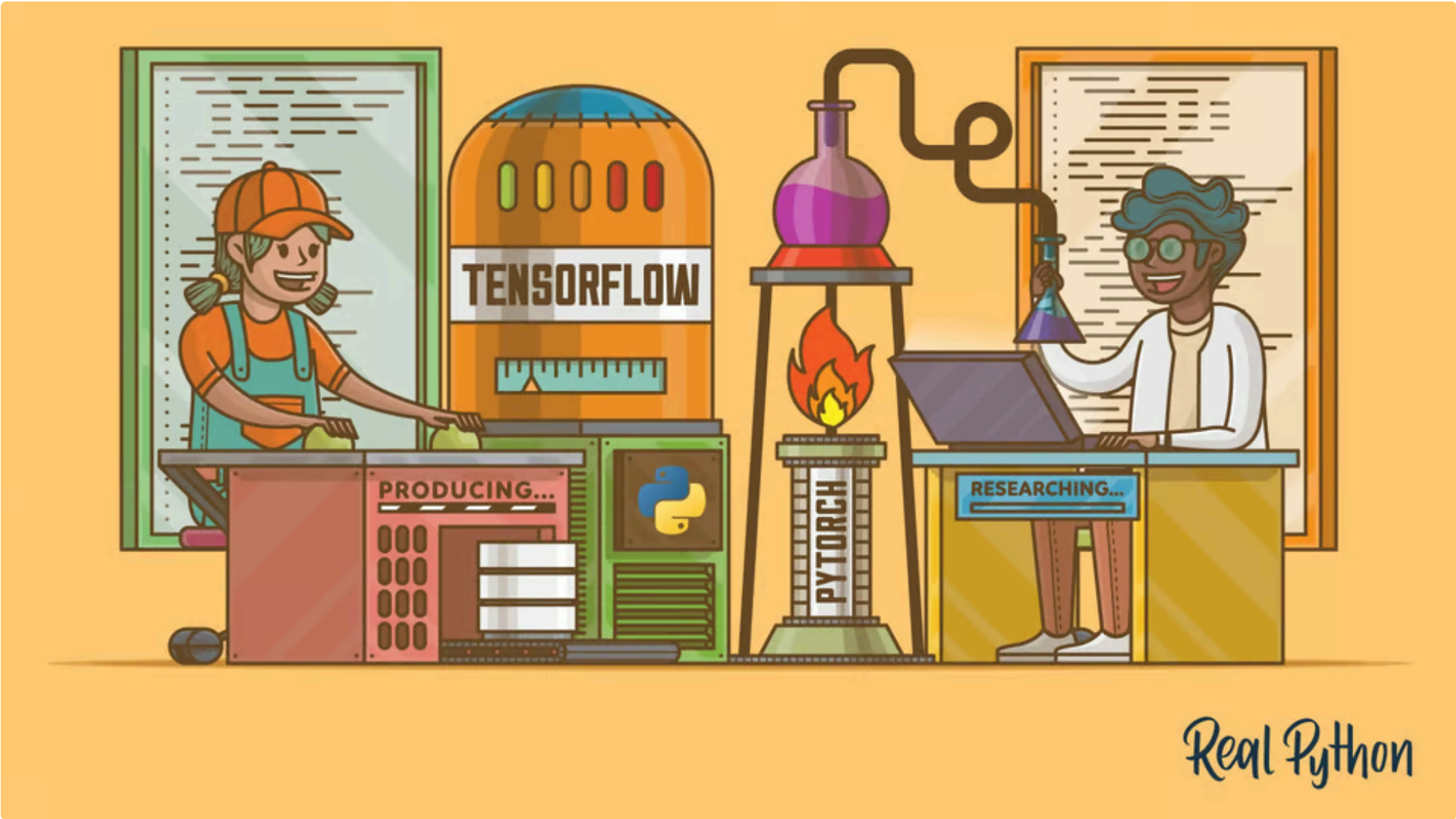




PyTorch vs TensorFlow for Your Python Deep Learning Project

by [Ray Johns](#) May 08, 2024 19m 1 Comment [advanced](#) [data-science](#) [machine-learning](#)

[Mark as Completed](#)

[Share](#)

Table of Contents

- [What Is TensorFlow?](#)
 - [Who Uses TensorFlow?](#)
 - [Code Style and Function](#)
 - [The Advantage of Using Keras](#)
 - [TensorFlow Special Features](#)
 - [TensorFlow Ecosystem](#)
- [What Is PyTorch?](#)
 - [Who Uses PyTorch?](#)
 - [Code Style and Function](#)
 - [PyTorch Special Features](#)
 - [PyTorch Ecosystem](#)
- [PyTorch vs TensorFlow Decision Guide](#)
 - [Style](#)
 - [Model](#)
 - [Data Type](#)
 - [Audio](#)
 - [Text](#)
 - [Images](#)
 - [Project Goal](#)
- [Conclusion](#)
- [Further Reading](#)

Clone a **175-agent** engineering team.

Clone the team →

Remove ads

Recommended Course

[Python Deep Learning: PyTorch vs Tensorflow](#) (21m)

PyTorch vs TensorFlow: What’s the difference? Both are open-source Python libraries that use graphs to perform numerical computations on data in deep learning applications. Both are used extensively in academic research and commercial code. Both are extended by a variety of APIs, cloud computing platforms, and model repositories.

If they’re so similar, then how do you decide which one is best for your project?

In this tutorial, you’ll learn:

- What the differences are between **PyTorch** and **TensorFlow**
- What **tools** and **resources** are available for each
- How to choose the **best option** for your specific use case

You’ll start by taking a close look at both platforms, beginning with the slightly older TensorFlow. Then, you’ll explore PyTorch and some considerations to help you determine which choice is best for your project. Let’s get started!

Get Your Decision Guide: [Click here to download the free decision guide](#) that will help you choose between PyTorch and TensorFlow for your Python deep learning project.

What Is TensorFlow?

TensorFlow was developed by Google and released as open-source in 2015. It grew out of Google’s homegrown machine learning software, which was refactored and optimized for use in production.

The name “TensorFlow” describes how you organize and perform operations on data. The basic data structure for both TensorFlow and PyTorch is a [tensor](#). When you use TensorFlow, you perform operations on the data in these tensors by building a [stateful dataflow graph](#), kind of like a flowchart that remembers past events.



Master Real-World Python Skills
With a Community of Experts

Level Up With Unlimited Access to Our Vast Library of Python Tutorials and Video Lessons

Watch Now »

Remove ads

Who Uses TensorFlow?

TensorFlow has a reputation for being a production-grade deep learning library. It has a large and active user base and a proliferation of official and third-party tools and platforms for training, deploying, and serving models.

After PyTorch was released in 2016, TensorFlow declined in popularity. But in late 2019, Google released [TensorFlow 2.0](#)—a major update that simplified the library and made it more user-friendly, leading to renewed interest among the machine learning community.

Code Style and Function

In TensorFlow 2.0, you can use [eager execution](#), which is how Python normally works. Eager execution evaluates operations immediately, so you can write your code using Python control flow rather than graph control flow. To see this in action, you’ll take a look at how you would multiply two tensors using both Tensorflow 1.0 and 2.0.

To start, here's an example of how to multiply tensors using TensorFlow 2.0. With eager execution, all you need is `tf.math.multiply()`:

Python



```
>>> import tensorflow as tf

>>> x = [[2., 4., 6.]]
>>> y = [[1.], [3.], [5.]]
>>> m = tf.math.multiply(x, y)

>>> m
<tf.Tensor: shape=(3, 3), dtype=float32, numpy=
array([[ 2.,  4.,  6.],
       [ 6., 12., 18.],
       [10., 20., 30.]], dtype=float32)>
```

In this code, you declare your tensors using Python's list notation, and `tf.math.multiply()` executes the **element-wise multiplication** immediately when you call it.

Before TensorFlow 2.0, you had to manually stitch together an [abstract syntax tree](#) by making `tf.*` API calls. You then had to pass a set of output tensors and input tensors to a `session.run()` call and manually compile the model.

A `Session` object is a [class for running TensorFlow operations](#). It contains the environment in which `Tensor` objects are evaluated and `Operation` objects are executed, and it can own resources like `tf.Variable` objects. The most common way to use a `Session` is as a [context manager](#).

To see how a `Session` is used in this way, here's an example of how you multiply two tensors using the old TensorFlow 1.0 method:

Python



```
>>> import tensorflow as tf

>>> tf.compat.v1.disable_eager_execution()

>>> x = tf.compat.v1.placeholder(tf.float32, name = "x")
>>> y = tf.compat.v1.placeholder(tf.float32, name = "y")

>>> multiply = tf.math.multiply(x, y)

>>> with tf.compat.v1.Session() as session:
...     m = session.run(
...         multiply, feed_dict={x: [[2., 4., 6.]], y: [[1.], [3.], [5.]]}
...     )
...     print(m)
[[ 2.  4.  6.]
 [ 6. 12. 18.]
 [10. 20. 30.]]
```

This code uses TensorFlow 2.x's `tf.compat` API to access TensorFlow 1.x methods and disable eager execution.

You first declare the input tensors `x` and `y` using [tf.compat.v1.placeholder](#) tensor objects. Then you define the operation to perform on them. Note that nothing has been calculated at this point.

Next, using the `tf.Session` object as a context manager, you create a container to encapsulate the runtime environment and do the multiplication by feeding real values into the placeholders with a `feed_dict`. Finally, still inside the session, you [print\(\)](#) the result.

Note: Keep in mind that because TensorFlow 1.0 has been deprecated, it probably isn't the best option for your future projects, and you should stick with using TensorFlow 2.0.

One other improvement in TensorFlow 2.0 is that it comes with a Python interface called [Keras](#). Keras has simpler APIs, rolls common use cases into prefabricated components for you, and provides better error messages than base TensorFlow.

The Advantage of Using Keras

To give you an idea of how Keras's layers can make your job easier, have a look at the code below. With only a few lines of code, you create a **regression model**—a model that learns patterns from a dataset of numbers and predicts unseen numbers based on those patterns. Assume your features are saved under `x_train`, and that each element of `y_train` corresponds to the target label for the corresponding sample in `x_train`:

Python

```
from keras.models import Sequential
from keras.layers import Dense

model = Sequential([
    Dense(10, activation="relu", input_shape=(32,)),
    Dense(1),
])

model.compile(optimizer="sgd", loss="mean_squared_error")

model.fit(X_train, y_train, epochs=10)
```

With only a few lines of code, you did quite a complex task! You created a two-layer regression model with a [stochastic gradient descent \(SGD\)](#) optimizer and [mean squared error](#) loss function. Take a look at the code snippet below to see how you can accomplish the same task without using Keras:

Python

```
import tensorflow as tf

class TwoLayerRegressionModel(tf.Module):
    def __init__(self):
        self.dense1_weights = tf.Variable(
            tf.random.normal([10, 10]), name="dense1_weights"
        )
        self.dense1_bias = tf.Variable(tf.zeros([10]), name="dense1_bias")

        self.dense2_weights = tf.Variable(
            tf.random.normal([10, 1]), name="dense2_weights"
        )
        self.dense2_bias = tf.Variable(tf.zeros([1]), name="dense2_bias")

    def __call__(self, x):
        x = tf.nn.relu(tf.matmul(x, self.dense1_weights) + self.dense1_bias)
        return tf.matmul(x, self.dense2_weights) + self.dense2_bias

model = TwoLayerRegressionModel()

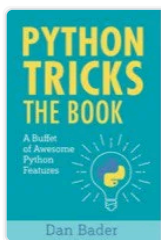
optimizer = tf.optimizers.SGD(learning_rate=0.01)
loss_function = tf.losses.MeanSquaredError()

def train_step(inputs, outputs):
    with tf.GradientTape() as tape:
        predictions = model(inputs)
        loss = loss_function(outputs, predictions)
    gradients = tape.gradient(loss, model.trainable_variables)
    optimizer.apply_gradients(zip(gradients, model.trainable_variables))
    return loss

for epoch in range(10):
    for i in range(len(X_train)):
        loss = train_step(X_train[i:i+1], y_train[i:i+1])
        print(f"Epoch {epoch}, Loss: {loss.numpy()}")
```

In this code snippet, you can see how it's training a two-layer regression model with an SGD optimizer and mean squared error loss function. However, without Keras, you need to do a lot more work! For example, unlike the last approach, you need a custom function for training your model and a custom for loop for calculating the error of your model.

If your use case doesn't fall into one of the [Core API applications](#), like building tools on top of TensorFlow or developing your own high-performance platform, you should prefer Keras.



“I wished I had access to a book like this when I started learning Python many years ago”
— Mariatta Wijaya, CPython Core Developer

[Learn More »](#)

 [Remove ads](#)

TensorFlow Special Features

TensorFlow has a large and well-established user base and a plethora of tools to help productionize machine learning. For mobile development, it has APIs for [JavaScript](#) and Swift, and [TensorFlow Lite](#) lets you compress and optimize models for Internet of Things (IoT) devices.

Another benefit of TensorFlow is that you can start using it quickly because of the wealth of data, pretrained models, and [Google Colab notebooks](#) that both Google and third-parties provide. Also, many popular machine learning algorithms and datasets are built into TensorFlow and are ready to use. In addition to the built-in datasets, you can access [Google Research Datasets](#) or use Google's [Dataset Search](#) to find even more.

As you've already seen, Keras makes it easier for you to get models up and running, so you can try out new techniques in less time. Indeed, Keras is the most used deep learning framework among the top five winningest teams on [Kaggle](#).

One drawback is that the update from TensorFlow 1.0 to TensorFlow 2.0 [changed so many features](#) that you might find yourself confused. [Upgrading code](#) is tedious and error-prone. Many resources, like tutorials, might contain outdated advice.

TensorFlow Ecosystem

Some highlights of the APIs, extensions, and useful tools of the TensorFlow extended ecosystem include:

- [TensorFlow Hub](#): A library for reusable machine learning modules.
- [Model Garden](#): An official collection of models that use TensorFlow's high-level APIs.
- [Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow](#): A comprehensive introduction to machine learning using TensorFlow.
- [TensorFlow.js](#): A library for machine learning in JavaScript.
- [KerasNLP](#): A natural language processing library that supports workflows built from modular components that have state-of-the-art preset weights and architectures.

What Is PyTorch?

PyTorch was developed by Facebook and was first [publicly released](#) in 2016. It was created to offer production optimizations similar to TensorFlow while making models easier to write. Because Python programmers found it so natural to use, PyTorch rapidly gained users, inspiring the TensorFlow team to adopt many of PyTorch's most popular features in TensorFlow 2.0.

Who Uses PyTorch?

PyTorch has a reputation for being more widely used in research than in production. However, since its release the year after TensorFlow, PyTorch has seen a sharp increase in usage by professional developers.

The [2023 Stack Overflow Developer Survey](#) list of the most popular “Other Frameworks, Libraries, and Tools” shows that 8.41% of developers use TensorFlow and 7.89% use PyTorch.

For context on PyTorch's growth, the [2020 Stack Overflow Developer Survey](#) indicated that 10.4 percent of professional developers use TensorFlow, while only 4.1 percent use PyTorch. Moreover, the [2018](#) survey reported that TensorFlow was used by 7.6 percent of developers, compared to just 1.6 percent for PyTorch.

As for research, PyTorch is a popular choice, and computer science programs like [Stanford's](#) now use it to teach deep learning.

Code Style and Function

PyTorch is based on [Torch](#), a framework for doing fast computation that is written in C. Torch has a [Lua](#) wrapper for constructing models.

PyTorch [wraps the same C back end](#) in a Python interface. But it’s more than just a wrapper. Developers built it from the ground up to make models easy to write for Python programmers. The underlying, low-level C and C++ code is optimized for running Python code. Because of this tight integration, you get:

- Better memory and optimization
- More sensible error messages
- Finer-grained control of model structure
- More transparent model behavior
- Better compatibility with NumPy

This means you can write highly customized neural network components directly in Python without having to use a lot of low-level functions. It’s also worth noting that PyTorch’s [eager execution](#), which evaluates tensor operations immediately and dynamically, inspired TensorFlow 2.0, so the APIs for both look a lot alike.

Converting NumPy objects to tensors is baked into PyTorch’s core data structures. That means you can easily switch back and forth between `torch.Tensor` objects and `numpy.array` objects.

For example, you can use PyTorch’s native support for converting NumPy arrays to tensors to create two `numpy.array` objects, turn each into a `torch.Tensor` object using `torch.from_numpy()`, and then take their [element-wise product](#):

Python

```
>>> import torch
>>> import numpy as np

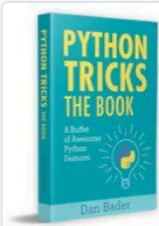
>>> x = np.array([[2., 4., 6.]])
>>> y = np.array([[1.], [3.], [5.]])

>>> m = torch.mul(torch.from_numpy(x), torch.from_numpy(y))

>>> m.numpy()
array([[ 2.,  4.,  6.],
       [ 6., 12., 18.],
       [10., 20., 30.]])
```

As you can see in this example, using `.numpy()` lets you print out the result of the multiplication, which is a `torch.Tensor` object converted to a `numpy.array` object.

The most important difference between a `torch.Tensor` object and a `numpy.array` object is that the `torch.Tensor` [class](#) has different methods and attributes, such as [backward\(\)](#) which computes the gradient, and [CUDA](#) compatibility.



“I don’t even feel like I’ve scratched the surface of what I can do with Python”

Write More Pythonic Code »

Remove ads

PyTorch Special Features

One of the main special features of PyTorch is that it adds a C++ module for [autodifferentiation](#) to the Torch backend. Autodifferentiation automatically calculates the gradient of the functions defined in `torch.nn` during [backpropagation](#).

By default, PyTorch uses **eager mode** computation. You can run a neural net as you build it, line by line, which makes it easier to debug. It also makes it possible to construct neural nets with conditional execution. This dynamic execution is more intuitive for most Python programmers.

PyTorch Ecosystem

Some highlights of the APIs, extensions, and useful tools of the PyTorch extended ecosystem include:

- [fast.ai](#): An API that makes it straightforward to build models quickly.
- [TorchServe](#): An open-source model server developed in collaboration between AWS and Facebook.
- [TorchElastic](#): A framework for training deep neural networks at scale using [Kubernetes](#).
- [PyTorch Hub](#): An active community for sharing and extending cutting-edge models.
- [TorchVision](#): A library dedicated to computer vision tasks that offers datasets, model architectures, and common image transformations.
- [TorchAudio](#): A library for audio processing and audio manipulation utilities.
- [TorchText](#): A natural language processing library that provides data processing utilities and popular datasets for the NLP field.

PyTorch vs TensorFlow Decision Guide

Deciding which library to use for a project depends on several factors. These include your personal style, the types of models and data you'll use, and your project goal. When you start your project with a little research on which library best supports these considerations, you'll set yourself up for success!

Style

If you're a Python programmer and you're used to the Python style of doing things, then PyTorch will feel easy to pick up. It works the way you'd expect it to, right out of the box.

On the other hand, more coding languages are supported in TensorFlow than in PyTorch, which has a C++ API. You can use TensorFlow in both JavaScript and Swift. If you don't want to write much low-level code, then Keras abstracts away a lot of the details for common use cases so you can build TensorFlow models without sweating the details.

Model

What models are you using? If you want to use a specific pretrained model, like BERT or DeepDream, then you should research what it's compatible with. Some pretrained models are available in only one library or the other, and some are available in both. The Model Garden and the PyTorch and TensorFlow hubs are also good resources to check.

Data Type

Another important question to ask yourself is about the data you'll be working with. What type of data do you need, or what kind of data do you have?

Depending on your dataset requirements and the specific data formats you're dealing with, PyTorch and TensorFlow both offer built-in functionality and extension libraries. There's no one-size-fits-all solution, so it makes sense to look at the data formats individually. The main data formats are:

- Audio
- Text
- Images

In the next few sections, you'll look at each of these data types and learn how both PyTorch and TensorFlow support them.

Audio

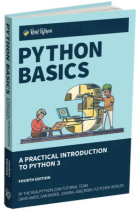
If your focus is audio data alone, then PyTorch is a good choice. PyTorch offers TorchAudio, which is an exclusive library for handling audio data. With TorchAudio, the complexity of audio processing for machine learning is reduced, thanks to necessary transformations like [resampling](#) and [short-time Fourier transform](#).

TorchAudio also helps you to access essential datasets, comprehensive feature extraction methods like [Mel spectrograms](#), and efficient file management for different audio types.

TensorFlow provides audio processing functionalities mainly through its core API, [tf.audio](#), and additional libraries like [TensorFlow I/O](#) and [TensorFlow Signal](#). The `tf.audio` API includes capabilities such as decoding [WAV files](#) and making spectrograms.

With TensorFlow Signal, you get more advanced options such as short-time Fourier transform and Mel spectrograms for deeper signal processing. It also includes audio data augmentation techniques like stretching sounds or changing pitch to build stronger models.

So, if you want an easy-to-use library for audio tasks like changing sample rates or creating Mel spectrograms, go with PyTorch's TorchAudio. It's made for audio and simplifies a lot of the work. But, if your project needs to handle audio along with other data types, or if you're after more advanced audio processing, TensorFlow with its `tf.audio`, TensorFlow I/O, and TensorFlow Signal might be the better fit.



Your Practical Introduction to Python 3 »

 [Remove ads](#)

Text

If you're working with text data, then TorchText is a great choice. TorchText is part of the PyTorch ecosystem and is designed to handle text data for natural language processing tasks. It offers tools for managing data loading, applying text transformations, and integrating with datasets.

This library simplifies tasks like text classification and language modeling by making vocabulary creation and tokenization more straightforward. TorchText also provides pretrained components, which let you prototype faster. Because of this, TorchText is really popular in experimentation, yet its modular architecture also offers adaptability for development requirements.

TensorFlow has two main companions that extend its capabilities to process text data efficiently—[TensorFlow Text](#) and [KerasNLP](#). KerasNLP provides high-level text processing modules that are available as layers or models. If you need access to lower-level tools, you can use TensorFlow Text. It includes advanced tokenization and preprocessing tools, as well as text classification and text generation.

So, if you're looking for something that's easy to use, quick to set up, and great for trying out new ideas, go with PyTorch's TorchText. It's perfect for getting NLP projects off the ground fast. But, if you need to do more detailed work with text, like fine-tuning how words are split up or handling complex text processing, then TensorFlow is the way to go.

Images

PyTorch's torchvision is made for working with images. It comes with ready-to-use datasets like [CIFAR](#), [MNIST](#), and [ImageNet](#) that you can use to test and train your models.

You also get access to pretrained models through `torchvision.models` for tasks like image classification, object detection, and segmentation. This is useful for getting a head start through [transfer learning](#), where you can tweak these models for your specific needs without having to start from zero.

TorchVision also includes all the important image transformations you'd need, like cropping, rotating, and normalizing. Plus, there are utilities that simplify the process of loading and showing images in batches, making your workflow smoother.

With TensorFlow's TF Image, part of TensorFlow's core, you get all the basics for resizing, cropping, and flipping images to get them ready for your models. Then there's [TensorFlow Datasets \(TFDS\)](#), which offers a huge selection of real-world image datasets that are easy to plug into your projects.

When it comes to pretrained models, TensorFlow Hub steps in with models that you can use right away, like [ResNet](#) and [MobileNet](#). You can save time by using these models to recognize patterns or objects in images so you don't have to start from scratch.

For image preprocessing, there's [Keras preprocessing layers](#) for preprocessing images directly within your model setups, making everything from adjusting image sizes to enhancing image quality a breeze.

When you’re deciding between PyTorch and TensorFlow for your work with images and computer vision tasks, it comes down to what your project needs. Sometimes, the exact tool or data you want to use might only be available in one of these options. Also, consider the ease of integration with your existing workflow and the availability of community support for your tasks, as these factors can significantly impact your project’s success.

Project Goal

The final question to ask yourself is about your project goal. Where will your model live? If you want to deploy a model on mobile devices, then TensorFlow is a good bet because of TensorFlow Lite and its Swift API. For serving models, TensorFlow has tight integration with Google Cloud, but PyTorch is integrated into TorchServe on AWS. If you want to enter Kaggle competitions, then Keras will let you quickly iterate over experiments.

Think about these questions and examples at the outset of your project. Nail down the two or three most important components, and either TensorFlow or PyTorch will emerge as the right choice.

Get Your Decision Guide: [Click here to download the free decision guide](#) that will help you choose between PyTorch and TensorFlow for your Python deep learning project.

Conclusion

In this tutorial, you’ve had an introduction to PyTorch and TensorFlow, looked at who uses them and what APIs they support, and learned how to decide whether PyTorch or TensorFlow is right for your project. You’ve seen the different programming languages, tools, datasets, and models that each one supports, and learned how to pick the one that is best for your unique style and project.

In this tutorial, you learned:

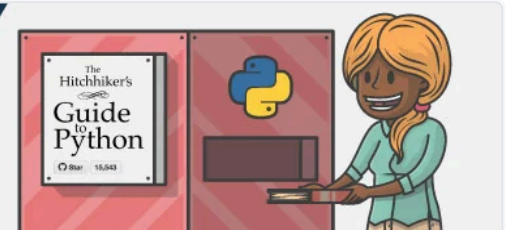
- What the differences are between **PyTorch** and **TensorFlow**
- How to use **tensors** to do computation in each
- Which platform is **best** for different kinds of projects
- What **tools** and **data** are supported by each

Now that you’ve decided which library to use, you’re ready to start building neural networks with them. Check out the links in Further Reading for ideas.

A Python Best Practices Handbook

python-guide.org

 [Remove ads](#)



Further Reading

The following tutorials are a great way to get hands-on practice with PyTorch and TensorFlow:

- [Practical Text Classification With Python and Keras](#) teaches you to build a natural language processing application with Keras.
- [Setting Up Python for Machine Learning on Windows](#) has information on installing PyTorch and Keras on Windows.
- [Pure Python vs NumPy vs TensorFlow Performance Comparison](#) teaches you how to do gradient descent using TensorFlow and NumPy and how to benchmark your code.
- [Python Context Managers and the “with” Statement](#) will help you understand why you need to use `with ... as session` in TensorFlow 1.0.
- [Generative Adversarial Networks: Build Your First Models](#) will walk you through using PyTorch to build a generative adversarial network to generate handwritten digits!

- [Python AI: How to Build a Neural Network & Make Predictions](#) will help you build a neural network from scratch using Python.
- The [Machine Learning in Python](#) learning path is a great source for more project ideas, like building a speech recognition engine or performing face recognition.

Mark as Completed

Share

Recommended Course

[Python Deep Learning: PyTorch vs Tensorflow](#) (21m)

Python Tricks

Get a short & sweet **Python Trick** delivered to your inbox every couple of days. No spam ever. Unsubscribe any time. Curated by the Real Python team.

```
1 # How to merge two dicts
2 # in Python 3.5+
3
4 >>> x = {'a': 1, 'b': 2}
5 >>> y = {'b': 3, 'c': 4}
6
7 >>> z = {**x, **y}
8
9 >>> z
10 {'c': 4, 'a': 1, 'b': 3}
```

Email Address

Send Me Python Tricks »

About Ray Johns



Ray is an avid Pythonista and writes for Real Python.

» [More about Ray](#)

Each tutorial at Real Python is created by a team of developers so that it meets our high quality standards. The team members who worked on this tutorial are:



[Aldren](#)



[Brenda](#)



[David](#)



[Geir Arne](#)



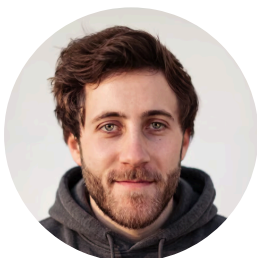
[Joanna](#)



[Jacob](#)

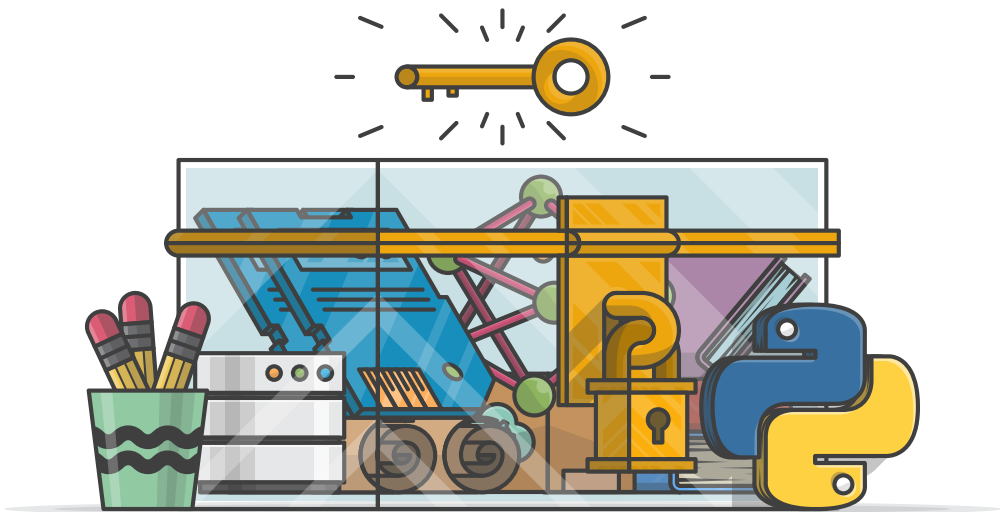


[Negar](#)



[Philipp](#)

Master Real-World Python Skills With Unlimited Access to Real Python



Join us and get access to thousands of tutorials, hands-on video courses, and a community of expert Pythonistas:

[Level Up Your Python Skills »](#)

What Do You Think?

Rate this article:



[LinkedIn](#) [Twitter](#) [Bluesky](#) [Facebook](#) [Email](#)

What’s your #1 takeaway or favorite thing you learned? How are you going to put your newfound skills to use? Leave a comment below and let us know.

Commenting Tips: The most useful comments are those written with the goal of learning from or helping out other students. [Get tips for asking good questions](#) and [get answers to common questions in our support portal](#).

Looking for a real-time conversation? Visit the [Real Python Community Chat](#) or join the next “Office Hours” [Live Q&A Session](#). Happy Pythoning!

Keep Learning

Related Topics: advanced data-science machine-learning

Related Learning Paths:

- Machine Learning With Python

Related Courses:

- Python Deep Learning: PyTorch vs Tensorflow

Related Tutorials:

- Generative Adversarial Networks: Build Your First Models
- Practical Text Classification With Python and Keras

Learn Python

Start Here

Learning Resources

Code Mentor

Python Reference

Python Cheat Sheet

Support Center

Courses & Paths

Learning Paths

Quizzes & Exercises

Browse Topics

Live Courses

Books

Community

Podcast

Newsletter

Community Chat

Office Hours

Learner Stories

Membership

Plans & Pricing

Team Plans

For Business

For Schools

Reviews

Company

About Us

Team

Mission & Values

Editorial Guidelines

Sponsorships

Careers

Press Kit

Merch



Privacy Policy · Terms of Use · Security · Contact

❤️ Happy Pythoning!

© 2012–2026 DevCademy Media Inc. DBA Real Python. All rights reserved.
REALPYTHON™ is a trademark of DevCademy Media Inc.

